

Work-in-Progress: From Generic Chatbots to Teacher-Configurable AI Tutors: The Design and Evaluation of Novedu in Secondary Education

Rainer Stropek and Andreas Probst

HTBLA (Höhere Technische Bundeslehranstalt) Leonding, Leonding, Austria
`r.stropek@htl-leonding.ac.at`

Abstract. Secondary-school students increasingly use general-purpose chatbots for learning, but these systems are rarely aligned with classroom context, teacher intentions, or a school’s privacy requirements. This work-in-progress paper presents *Novedu*, a teacher-configurable AI-tutoring platform developed at HTBLA Leonding, an Austrian technical upper-secondary school. Our position is that a school cannot realistically align large language model (LLM) tutors by training models, but can align them institutionally by giving teachers ownership of configuration, curricular grounding, deployment, privacy defaults, and evaluation. Tutors are therefore assembled from reusable, teacher-authored modules and served through in-region, open-weight models run by a research partner. A requirements survey ($n = 318$) yields four central requirements (curricular grounding, factual reliability, configurable scaffolding, and aggregated oversight), each traced to a design response; a reproducible evaluation harness gives preliminary one-topic evidence that an in-region open-weight model can be competitive with frontier baselines, while depending strongly on teacher-authored prompt detail. The platform and classroom evaluation remain ongoing.

Keywords: generative AI in education · teacher-configurable AI tutors · pedagogical alignment · in-region open-weight LLMs · secondary education

1 Introduction

Generative AI has become an everyday tool in secondary education. At our school adoption is near-universal: 94% of students have used an AI tool, 84.5% regularly (Section 3). The opportunities and risks are well rehearsed [2]; what remains unresolved for an individual school is not *whether* students use AI, but *how* the institution can channel that use toward learning while meeting its legal and pedagogical obligations.

A general-purpose chatbot is a poor default for two reasons. Pedagogically, used as-is an LLM tends to provide immediate answers rather than guiding students through problem solving [4], and it knows neither the class’s current topic nor the teacher’s intentions. Institutionally, sending student data to commercial

cloud LLMs raises data-protection obligations under the GDPR and, for some educational uses such as evaluation or steering of learning, governance obligations under the EU AI Act, and it gives teachers no oversight of its use in their subject.

This paper presents *Novedu*, a teacher-configurable AI-tutoring platform built at HTBLA Leonding, an Austrian technical upper-secondary college (an *HTL*) with a strong informatics profile. Our central position is that a school cannot realistically align tutors by *training* models, but it can align them *institutionally*: by giving teachers ownership of configuration, curricular grounding, deployment, privacy defaults, and evaluation. Pedagogical alignment thus becomes a configuration and authoring problem owned by teachers, not a model-training problem owned by ML engineers.

We make three contributions: (1) an initial requirements evidence base from a school-wide survey, traced explicitly to design decisions; (2) a teacher-configurable architecture in which tutors are assembled from reusable, teacher-authored modules served over in-region, open-weight models; and (3) a reproducible evaluation path with preliminary results on tutoring behaviour. We organise the paper around three design questions. **DQ1**: what do students and teachers at a technically oriented secondary school require of an institutional AI tutor? **DQ2**: how can a school align tutors with those requirements through teacher-owned configuration rather than model training? **DQ3**: can a small, in-region open-weight model deliver the intended Socratic tutoring behaviour, and how much prompt detail must teachers provide? Consistent with the work-in-progress category, the platform and broader evaluation remain ongoing.

2 Related Work and Positioning

Recent work converges on one idea, *pedagogical alignment*: a general-purpose LLM, used as-is, behaves as a generic information provider that hands over answers, whereas effective teaching requires scaffolded, Socratic guidance [4]. The works differ in *how* they close that gap. One family aligns models through training: Sonkar et al. [4] use preference learning to make a model guide rather than answer, and contribute metrics that quantify scaffolding versus direct answers. A second family aligns through prompt or configuration: Gabrovšek and Rihtaršič [1] deploy a configurable generative-AI tutor in an undergraduate STEM lab and report that structured, declarative prompt styles were rated highest by students. The broader effectiveness of (non-LLM) intelligent tutoring systems is well established, with meta-analytic gains around 0.66 SD over conventional instruction [3].

Novedu adopts the configuration route, the most realistic lever available to a secondary school, which cannot curate preference datasets or fine-tune models but can author prompts and use in-region open-weight models. Our contribution is largely orthogonal: where this literature studies *the tutor*, we study *the school's path to fielding tutors*, an institutional embedding (teacher ownership, curricular fit, privacy, in-region inference, evaluation) that international guidance increas-

ingly argues schools must address as a whole. For evaluation we build on work measuring the *pedagogical* quality of responses rather than answer correctness [5] and on the LLM-as-a-judge paradigm and its biases [6].

3 Requirements and Design Process

From exploratory use to a scope-reduced platform. Novedu emerged from a requirements-driven progression rather than an up-front specification. A vision document committed to teacher-edited prompts, per-tutor model choice, math/code rendering, open-source components, and self-hostable hosting. Students then composed existing open-source products (Open WebUI with a model gateway, school single sign-on, per-user budgets) into a working environment; this prototype was an instrument for learning where a generic chatbot UI falls short pedagogically. It was not evaluated as a product in isolation; its use informed the requirements process and motivated the scope reduction reported here. Guided by a senior product owner (the head of the computer-science department), the team produced a requirements specification and a full UX prototype of an institutional platform; a cross-disciplinary teacher panel received it well but judged it too large to deploy quickly, prompting a scoping decision: reduce the vision to a minimal deployable tutor workflow, deferring institutional features (admin consoles, user/group management, cost dashboards). Across the project, students built the Open WebUI prototype, the requirements and UX artifacts, and parts of the continuation; the final minimum viable product (MVP) seed was created in a teacher-led workshop and continued cooperatively.

Requirements survey. Running alongside this process, a two-variant questionnaire (students; teachers) was fielded in supervised in-class sessions, yielding 318 valid responses (284 students, 34 teachers) on a 4-point Likert scale with no neutral midpoint (theoretical mean 2.5). The most important caveat is sample bias: the school is informatics-heavy (72.9% of student respondents in the IT branch), so the findings describe a technically literate, AI-fluent population and do not generalise directly to general-education schools. Four mutually reinforcing requirements dominate across both groups. *Curricular grounding* is the strongest single signal: the top student item is that the tutor know their concrete lesson content ($M = 3.53$, uniform across branches). *Factual reliability* is critical on both sides: wrong answers are students’ most frequent objection and teachers’ top property (84.8%). *Configurable scaffolding* beats any fixed mode: students’ top explanation choice is “it depends, let me choose” (52%), and teachers want help behaviour configurable per workspace. *Non-surveillance oversight* is required by both: students dislike teachers reading even anonymised chats (57%), while teachers want aggregated statistics (75.8%) over individual chats (6%). Two adoption findings frame the rest: teachers are willing but under-prepared (only 6% feel well-informed on the legal framework; training demand is among the highest items, $M = 3.27$), and willingness to author tutors is conditional on low effort.

Table 1. Requirements-to-design traceability. Survey signals (Section 3) drive design responses (Section 4); remaining gaps motivate future work (Section 6).

Survey signal	Design implication	Novedu response	Remaining gap
Tutor must know lesson content ($M=3.53$)	Curricular grounding > generic chat	Teacher-authored topic / tutor modules	Retrieval over LMS materials not yet built
Wrong answers top objection	Reliability must be evaluated	SocraticTutorEval harness (Sec. 5)	No classroom learning-outcome data yet
Want configurable help style (52%)	One Socratic mode is too rigid	Reusable YAML behaviour fragments	Authoring must become easier
Reject read-along; want aggregate stats	Oversight must be non-surveillance	Anonymous defaults, aggregate code stats	Behavioural evaluation design needed
Want training and templates	Adoption depends on low effort	Prompt library, starter tutors, AI authoring aids	Teacher training still future work

Table 1 traces each requirement to a design response in Novedu and to the gap that remains, making the design decisions answerable to the evidence. Section 4 details these responses.

4 Novedu Design

4.1 Teacher-Configurable Tutors as Modular Configuration

A single idea organises the architecture: a tutor is a plain, human-readable file, and the application is a configuration-driven layer around it, so pedagogy lives in teacher-authored configuration, not in application code or model weights. Two kinds of YAML files are composed. *Fragment libraries* are centrally maintained, reusable prompt modules (a persona, ground rules such as a “never give away the solution” Socratic policy, a topic description), each versioned and parameterised by an input schema. *Tutor files*, written per subject, select fragments, fill in their inputs, add final instructions, and reference a model. A teacher therefore writes only a short, structured definition (Fig. 1).¹ At request time a framework-agnostic pipeline fetches, validates, and assembles the fragments into a system prompt for a single agent answering over an in-region open-weight model.

The same behaviour module thus serves mathematics, German, or programming teachers alike.

¹ Real tutor and fragment files are available in the project repository: <https://github.com/Teaching-HTL-Leonding/novedu-chat-mvp>.

```

id: linked-lists
name: Linked-list tutor
llm:
  model: gemma-4-31B-FP8
prompt:
  fragments:
    - url: fragments/socratic-no-solution.yaml
    - url: topics/linked-lists.yaml
  instructions: >
    Focus on traversal, insertion, deletion, and edge cases.
    Ask one guiding question at a time.

```

Fig. 1. A complete tutor file. It stays short because reusable fragments (here a Socratic no-solution policy and a linked-lists topic) carry the detailed behaviour; the teacher sets only the topic focus and a few rules. Full examples are in the project repository.

This pedagogical content is produced by a cross-disciplinary, teacher-owned effort: a shared, version-controlled library of tutor fragments (such as the Socratic “no-solution” policy referenced above) authored by teachers of German, English, mathematics, biomedicine, and programming. It is the clearest single artifact of institutional embedding: the platform’s pedagogy is shaped by teachers across faculties, not by the development team alone.

Authoring usability. A first presentation to non-technical teachers showed that authoring YAML by hand is a real usability barrier, matching the survey’s low-effort precondition. We address this without abandoning the plain-file model: a JSON Schema gives autocompletion and inline validation in ordinary editors, an AI authoring aid turns prose into a valid tutor definition, and a command-line verification tool (with an AI skill) returns precise errors. A graphical editor that hides YAML entirely is the next step (a final-year student project).

4.2 One Abstraction, Several Activities

The configurable-file idea generalises beyond chat: a teacher mints a short *code* pointing at an activity definition, and one access-gating and statistics mechanism serves whichever activity the code names. Table 2 summarises the four types; tutors carry the most pedagogical weight, while quizzes, writing, and the coding proxy show the abstraction generalises (anonymity is the default except where graded writing requires attribution).

4.3 Self-Hosting and Privacy Defaults

Institutional fit rests on a few deployment decisions. Inference avoids commercial cloud LLMs: open-weight, quantized models (the ≈ 27 –31B range) are served behind an OpenAI-compatible API, addressed server-side only, by an in-region GPU server operated by the research centre SCCH (Software Competence Center Hagenberg) through its *LLM:2go* offering, made available to the school at no

Table 2. Activity types sharing the configurable-file abstraction. Each is defined by a teacher-authored file; the application is a configuration-driven layer around it.

Activity	What the teacher configures	Default
Tutor	A Socratic-by-default persona, rules, topic, and model; renders mathematics and code	Anonymous
Quiz	Open-ended questions plus a server-only grading rubric; a stateless grader scores free-text answers and gives feedback	Anonymous
Writing	A task and a coaching persona for a split-screen coach that can read but not edit the draft	Attributed
Coding	A system prompt injected into an OpenAI-compatible proxy for students’ coding agents	Anonymous

cost through a government-funded research project.² Identity is the school’s own (single sign-on, teacher actions gated by group membership), and privacy is the default: activities are anonymous unless a teacher opts out, the quiz grader stores nothing, telemetry carries no message content, and chat history is not retained. These choices support GDPR- and AI-Act-aware deployment by avoiding commercial, data-collecting LLM services and keeping processing in-region; SCCH is nonetheless a separate processor, and a full compliance assessment is outside this paper’s scope. Building the platform is itself a cooperative student software-engineering project under teacher direction, a secondary computing-education contribution we do not develop here.

5 Preliminary Evaluation

The survey measures stated preferences. To measure *behaviour*, whether a configured tutor actually withholds the answer, we built *SocraticTutorEval*: given a partial student–tutor conversation ending on a student turn, it generates the next tutor reply with several LLMs and has an LLM judge score each against a per-scenario rubric (passing requires no negative-criterion violation and at least 70% of positive criteria met). A failing reply does not indicate a wrong answer; it indicates a departure from the behaviour the teacher specified, primarily abandoning the Socratic stance by volunteering the solution. We use it to probe DQ3 on one programming topic (linked lists): three system prompts of increasing length for the same tutor (a 6-line *minimal* prompt of two sentences, an 84-line *short*, and a 172-line *full* prompt), across one in-region open-weight model (the

² LLM:2go is SCCH’s general-purpose, open-source, self-hostable LLM offering (not school-specific) for organisations that cannot use public-cloud LLMs: <https://www.scch.at/aktuelles/llm2go>.

Table 3. Pass rate (%) by model and prompt length on the linked-lists scenarios. Higher prompt detail helps every model; the in-region SCCH model is competitive with frontier baselines and descriptively most robust to a terse prompt. Each cell contains only 36 judged conversations, so rates should be read descriptively: one additional pass shifts a cell by 2.8 points.

Model	6-line	84-line	172-line
gemma-4-31B-FP8 (SCCH, in-region)	47.2	55.6	61.1
gpt-oss-120b (open weight, medium)	33.3	52.8	61.1
gpt-5.4-mini	27.8	55.6	61.1
gpt-5.4 (medium reasoning)	33.3	47.2	55.6
gpt-5.4 (high reasoning)	27.8	50.0	47.2

SCCH endpoint) and four frontier baselines (Table 3). Each cell aggregates 36 judged conversations (12 scenarios $\times$ 3 generated replies), each scored once by **gpt-5.4** at high reasoning effort.

Two findings stand out. First, *prompt detail matters substantially*: moving from the full prompt to the two-sentence minimal prompt lowers the pass rate for every model, by 14 points for the in-region model and up to 33 points for **gpt-5.4-mini**. Teacher-authored prompt detail is therefore consequential (DQ2), reinforcing the need for authoring support (Section 4.1). Second, on this Socratic-adherence metric a small *in-region open-weight* model is viable (DQ3): **gemma-4-31B** matches the best frontier configuration on the full prompt (61.1%) and is descriptively the most robust to a terse prompt (47.2%, versus $\leq 33.3\%$); higher reasoning effort did not help. The frontier models do not score lower because they are weaker; they are more capable, and that works against them, since their characteristic failure is over-responding, handing the student a full answer and breaking the Socratic constraint. It proved easier to steer a small model into Socratic behaviour than to restrain a large one from over-helping. These preliminary results make in-region open-weight inference plausible for further classroom testing, but they do not establish general superiority over commercial models. Notably, these prompts predate Novedu’s fragment model; the burden of maintaining a long prompt by hand motivated the reusable-fragment design (Section 4.1), an instance of evaluation feeding back into design.

These results are preliminary: a single topic, small per-cell counts, and an LLM judge whose biases and calibration require scrutiny [6] and may diverge from human teachers on helpfulness [5]. The harness has been shared with the teacher group and first cross-teacher experiments have begun; validating the judge against independent teacher ratings on a stratified sample of replies is part of the ongoing, multi-subject analysis. Model IDs, snapshots, decoding parameters, the judge, and prompts are archived with the harness for reproducibility.

6 Limitations and Next Steps

The platform is a learning prototype: its interface is functional rather than polished, it is not yet usable on smartphones (a real gap, since phones are a primary student device), and it is single-tenant. The evaluation is correspondingly bounded: the survey is single-site, supervised, and self-report only, with an informatics-heavy sample that limits external validity; the harness covers one topic and depends on an LLM judge that needs calibration; and we have no classroom learning-outcome data yet.

The most important gap is that our strongest requirement outruns the implementation: students demand that the tutor know their concrete lesson material, yet the MVP supplies this only through teacher-authored prompt modules, not retrieval over the school’s learning-management system. Closing that gap is the priority next step. Equally central is adoption: a teacher-configurable platform works only if teachers can and will configure it, so the graphical authoring editor and a teacher-training effort (the survey’s clear demand) are conditions for success, not minor items. Further work includes a behavioural, in-classroom evaluation on aggregated, anonymised data and shipping the coding proxy; the cooperative-development model and the configuration-over-training architecture may then transfer to other computer-science schools.

7 Conclusion

Novedu is a design case of how a school can move from exploratory AI use toward a teacher-configurable, privacy-aware tutoring infrastructure. Rather than aligning models by training, it gives teachers ownership of configuration: tutors are assembled from reusable, plain-text modules and served over in-region, open-weight models, with anonymous defaults and aggregated oversight. A school-wide survey supports four core requirements, each traced to a design response, and a reproducible harness gives early one-topic evidence that a small in-region model was competitive with frontier baselines while depending strongly on prompt detail. The platform and its evaluation remain ongoing; the contribution is a design case, a requirements evidence base, and an evaluation path for institutionally embedded AI tutoring.

References

1. Gabrovšek, R., Rihtaršič, D.: Custom generative artificial intelligence tutors in action: An experimental evaluation of prompt strategies in STEM education. *Sustainability* 17(21), 9508 (2025)
2. Kasneci, E., Seßler, K., Küchemann, S., et al.: ChatGPT for good? on opportunities and challenges of large language models for education. *Learning and Individual Differences* 103, 102274 (2023)
3. Kulik, J.A., Fletcher, J.D.: Effectiveness of intelligent tutoring systems: A meta-analytic review. *Review of Educational Research* 86(1), 42–78 (2016)

4. Sonkar, S., Ni, K., Chaudhary, S., Baraniuk, R.G.: Pedagogical alignment of large language models. In: Findings of the Association for Computational Linguistics: EMNLP 2024. pp. 13641–13650. Association for Computational Linguistics (2024)
5. Tack, A., Piech, C.: The AI teacher test: Measuring the pedagogical ability of Blender and GPT-3 in educational dialogues. In: Proc. 15th International Conference on Educational Data Mining (EDM 2022). pp. 522–529 (2022)
6. Zheng, L., Chiang, W.L., Sheng, Y., et al.: Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In: Advances in Neural Information Processing Systems 36 (NeurIPS 2023) (2023)